

# Sample-Efficient Reinforcement Learning for Tool-Using Video Agents

Ismael Arechiga Duran

iaredur@stanford.edu

[github.com/DonIsmaelito/VideoAgent-RL](https://github.com/DonIsmaelito/VideoAgent-RL)

## Abstract

I wanted to know how much of a tool-using video agent’s behavior actually has to be learned. Recent systems wrap a large language model in a set of tools and train the whole controller end to end with reinforcement learning, usually a method called GRPO. That works, but it is expensive, and it learns several decisions at once, so it is hard to say what the learning is really buying. So I built the smallest version of the problem I could. I froze the parts that look at the video and pick an answer, and let the agent learn only the controller, the part that decides what to do next. Then I dropped four controllers into the same setup, from a purely random one up to PPO, and watched what changed. The honest answer is less than I expected. Even a purely random controller nearly ties the learned ones on accuracy, the three learners land within a fraction of a point of each other, and the simplest of them gets there about three times sooner. Almost all of the value worth learning is in when to stop, not which tool to call, because the frozen answer function barely cares which frames it gets. So what the learning really buys is efficiency, not better answers. The takeaway I came away with: for this kind of agent a simple learned stopping rule recovers most of what heavy reinforcement learning gets credited for, and the real bottleneck is the part that reads the frames.

## 1 Introduction

Long-form video understanding is hard for a simple reason: the answer to a question usually lives in a few seconds of a clip that can run for minutes or hours. One popular fix is to make the model agentic. Instead of feeding every frame to one network, an agent looks at the video a piece at a time. It pulls up some frames, sometimes captions them, decides whether it has seen enough, and only then answers. VideoAgent (Wang et al., 2024) is a clean example and does well on the NExT-QA benchmark using GPT-4 as its reasoner. When I read the work that followed, the trend was clear: people are making the reasoner learned instead of hand-prompted, training it with reinforcement learning (Zhang et al., 2026; Tian et al., 2025; Zhou et al., 2025), where the model learns by trying actions and getting a reward when it does well. The method of choice is GRPO (Shao et al., 2024), run on top of ordinary supervised training.

This works, but two things bothered me. First, it is expensive. GRPO needs many attempts per question, every attempt calls a large model, and the whole controller is trained at once, so any improvement is credited to all of its decisions at the same time. Most people, including me, do not have that kind of compute. Second, and more interesting to me, none of these papers say which decision the learning is actually helping. An agent picks which tool to call, where to look, and when to stop, and there is no reason those should be equally hard. I wanted to pull them apart.

So I set up the most controlled version of the problem I could. I froze the parts that see and answer, and let the agent learn only the controller, the part that decides what to do next. The input is a multiple-choice question plus a bank of per-frame image features for the video, computed once with CLIP (Radford et al., 2021), a model that puts images and short pieces of text in the same vector space so you can compare them with a cosine similarity. A controller issues a short sequence of tool actions and then commits to an answer. Because the part that turns gathered evidence into an answer is fixed and shared by every method, any difference I see between controllers is the controller’s

doing and nothing else. Then I put classical learners in the same ring as PPO, and I switched off parts of the action space to isolate one decision at a time.

Three things came out of this. A simple bandit matches PPO on quality while learning far faster. The stopping decision, not tool choice, is where the learnable value sits. And accuracy is capped by the frozen answer function rather than the controller, so past a point a smarter policy cannot help. None of the pieces here are new on their own. What I had not seen before is someone (i) putting classical reinforcement learning head to head with the modern recipe on a tool-using video agent, (ii) freezing a cheap shared answer function so the comparison is about the policy and nothing else, and (iii) running an ablation that separates the value of choosing a tool from the value of choosing when to stop.

## 2 Related Work

### 2.1 Video agents without learning

VideoAgent (Wang et al., 2024) and VideoAgent2 (Yang et al., 2025) use a language model as the controller. It retrieves frames and reasons over them in a loop, but the loop itself is hand-designed or driven by prompts. These systems are accurate and need no training. For the same reason, though, they cannot get better from experience: there is nothing in them that learns.

### 2.2 Reinforcement learning for video agents

EVA (Zhang et al., 2026), Ego-R1 (Tian et al., 2025), and ReAgent-V (Zhou et al., 2025) make the controller learned, training it with reinforcement learning on top of supervised and preference fine-tuning, using GRPO. EVA shows the scale this involves: it builds on a seven-billion-parameter vision-language model and runs a three-stage pipeline of supervised fine-tuning, preference optimization, and GRPO, turning what they call a passive video recognizer into an active watcher that decides what to watch, when, and how. They show convincingly that learning helps. But they all jump straight to the modern recipe. None of them report a classical reinforcement-learning baseline, and all of them train the whole policy at once, so tool choice and stopping stay tangled together. They also each evaluate in their own setup, which makes it hard to read off how much the learner by itself is contributing. Those gaps are what I wanted to fill.

### 2.3 Classical reinforcement learning

The learners I reach for are old and well understood. UCB1 (Auer et al., 2002) is a bandit rule from 2002. REINFORCE (Williams, 1992) is the original policy-gradient method, from 1992. PPO (Schulman et al., 2017) is the modern deep reinforcement learning workhorse. All three are simple and sample efficient, but as far as I can tell nobody has tried them on a tool-using video agent, which is exactly the comparison I wanted to see.

## 3 Datasets

My main testbed is NExT-QA (Xiao et al., 2021), a video question-answering benchmark where each question has five options, so blind guessing scores 20%. Its questions come in three flavors: causal (why something happened), temporal (what happened before or after), and descriptive (what is in the scene). I follow the official train and validation split. I train on a 1,000-video sample of the training set (8,751 questions) and report on the full validation set (570 videos, 4,996 questions, of which 2,607 are causal, 1,612 temporal, and 777 descriptive). To check whether anything I learn transfers, I also use the public 500-question slice of EgoSchema (Mangalam et al., 2023), a set of roughly three-minute first-person videos. I never train on EgoSchema and only test on it, which is what zero-shot means. Its published answer key matched my parsed labels exactly, which was a relief.

For features I keep things cheap and fixed. I sample 32 frames evenly across each video and run them through a frozen CLIP image encoder, which turns each frame into a vector of 768 numbers. I encode the question, the five options, and three reworded versions of the question (one each aimed at objects, actions, and timing) with the matching CLIP text encoder, so frames and text live in one

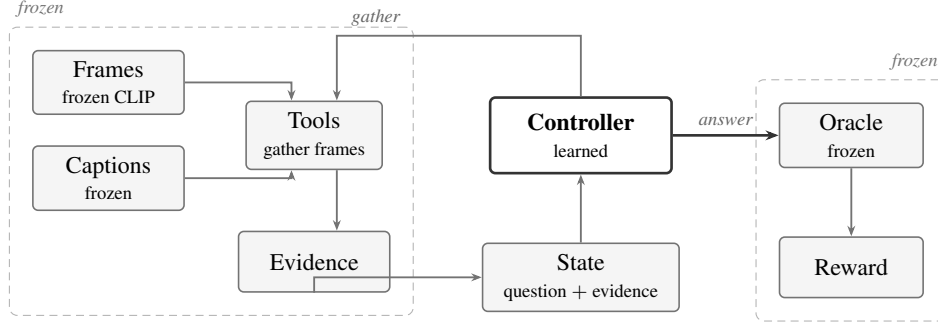


Figure 1: The agent as a Markov decision process with one learned part. Everything inside a dashed gray box is frozen; only the Controller, shown in bold, is trained. The frozen tools gather frames into the evidence, which with the question forms the state the controller reads. At each step the controller makes one choice: gather more, or answer and stop. With perception and answering fixed, the only thing worth learning is when to stop.

shared space and a cosine similarity between them means something. I also caption six frames per video with BLIP-2 (Li et al., 2023), a captioning model, and embed those captions the same way. Everything is computed once and cached. None of these vision models are ever fine-tuned. The only weights that learn anything in the whole system belong to the controller.

## 4 Method

### 4.1 Environment

I wrote the agent’s loop as a Markov decision process, which is just the standard way to describe a step-by-step decision problem in terms of states, actions, and rewards. Figure 1 lays out the whole loop, and its point is the division of labor: everything drawn in gray is frozen, and the controller, the one box drawn in bold, is the only part that learns. I borrowed how EVA (Zhang et al., 2026) factors the state. At any moment the state is the question, the evidence gathered so far, and the bank of frames, which the controller reads as a single 1,553-dimensional observation built from the question embedding, a pooled summary of the evidence, and a handful of scalars such as its confidence, the best-matching frame, the focus window, the per-action counts, and the budget that remains. From that observation the controller picks one of eight actions. Seven of them gather: three retrieval actions each pull in the few frames that best match a reworded version of the question, an overview resets the focus to the whole video, a zoom and a widen move the focus window in or out, and a caption action reveals the cached captions; each pools new frames into the evidence and loops back, the loop in Figure 1. The eighth action answers and ends the episode, the answer arrow in the figure. Because the perception on the left and the answerer on the right are both fixed, the controller’s real choice on every step reduces to the fork the figure draws: keep gathering, or stop and answer.

### 4.2 The frozen reader

Every controller hands off to the same fixed answer function. I call it the oracle only because it is frozen and shared, not because it is clever. Given the pooled evidence vector  $e$  and the option vectors  $o_j$ , it picks

$$\hat{y} = \arg \max_j \cos(e, o_j) + \alpha \cos(e, [q; o_j]),$$

and when no evidence has been gathered yet it falls back to comparing the question straight to the options,  $\arg \max_j \cos(q, o_j)$ . This is deliberately a weak reader. Freezing it is the whole point: if every method answers the same way, any difference between methods has to come from the controller, not from one of them getting a better reader.

### 4.3 Why a cached CLIP setup

The whole environment runs on cached CLIP vectors rather than calling a model over the video at each step, and that one choice shapes everything, so it is worth saying why. Part of it is the control I just described: freezing one cheap reader for every method makes any difference the controller’s doing. The rest is cost. The paper is asking whether you need GRPO-scale compute at all, so the testbed has to be cheap, and encoding the 32 frames and the question once with CLIP and caching them turns every step into a few dot products on a CPU. That is how I ran millions of episodes for under fifteen dollars; calling a real model on every step would have cost the very thing I am questioning. The honest catch, which the results make plain, is that this buys cheapness at the price of a weak reader. CLIP barely cares which frames it gets, so the problem comes out shallow on accuracy and the only thing left to learn is when to stop. I would not build it this way to win a benchmark. I built it this way to isolate the controller as cheaply as possible, and the weakness it exposes points straight at the fix, a stronger frozen reader.

### 4.4 Reward

I score each episode simply. Every tool call costs a small penalty  $c = 0.03$ , and at the end the agent gets 1 for a correct answer and 0 for a wrong one, so the return for an episode is

$$R = \mathbf{1}[\text{correct}] - c \cdot (\text{number of tool calls}).$$

The agent has a budget of ten calls; if it has not answered by then, it is forced to. This gently pushes it to be both right and quick.

### 4.5 Decision-isolation ablation

To find out which decision matters, I run the same controllers under three settings. In the joint setting all eight actions are available, which is the real problem. In tool-choice-only, the environment decides when to stop for the agent, using a simple confidence-or-budget rule, and the agent only picks tools. In stopping-only, the environment runs a fixed script of tools and the agent only chooses between continuing and answering. Comparing the three tells me directly which decision the learning was buying.

## 5 Experiments, Results, and Discussion

### 5.1 Setup and metrics

I report accuracy (overall and by category), the average number of tool calls, and the return. Every evaluation is greedy and deterministic, run on the held-out validation videos, which share no videos with training. For hyperparameters I used standard defaults and did not tune much: UCB1 with an exploration constant of 1.4 for 20k episodes; REINFORCE at learning rate  $10^{-3}$  in batches of 32 episodes for 40k episodes; PPO with the usual settings (2048-step rollouts, batch size 64, ten epochs, discount 0.99) for 500k steps. The whole study cost under \$15, with the one-time feature extraction on a rented T4 GPU and all the actual training on a CPU. That cheapness is part of the point.

### 5.2 Main results

The first thing I checked is whether learning helps at all, my first question. Table 1 says yes, but in a specific way. The three learners land within a tenth of a point of each other on accuracy, and they answer in one or two tool calls where the random controller burns through five. So next to random, learning lifts the return from about 0.33 to about 0.48 and cuts the calls roughly fivefold. The bandit and REINFORCE settle on a single call; PPO is a little less tidy, settling around one and a half calls, which is why its return sits a couple of points below the other two. What it does not do is separate the methods from each other on accuracy, and that is the part worth dwelling on. The random controller sits at about 0.49, basically tied with the learners, and once I split its runs by how much it gathered the reason got sharper and a little humbling. About 12.5% of the time random answers before gathering anything, since answering is one of eight actions, and those runs score 0.29, the no-evidence prior. The other 87.5% gather something first and score 0.52, if anything a hair above the learners. So on accuracy the learned policies do not beat random at all. They actually

Table 1: NExT-QA validation (4,996 questions) and EgoSchema. Accuracy, by-category accuracy, return, and mean tool calls. All numbers are means over three random seeds; the standard deviation is at most 0.007 on every accuracy and return cell, so the learned policies are statistically indistinguishable on accuracy. VideoAgent and EVA (Zhang et al., 2026) are published references, not run in my environment. EVA does not evaluate on NExT-QA or EgoSchema, so its entry (†) is its VideoMME accuracy on long-video benchmarks (GRPO on Qwen2.5-VL-7B), shown for context and not directly comparable.

| Policy            | NExT-QA validation |        |          |        |        |       | EgoSchema |       |
|-------------------|--------------------|--------|----------|--------|--------|-------|-----------|-------|
|                   | Acc                | Causal | Temporal | Descr. | Return | Calls | Acc       | Calls |
| Random            | 0.488              | 0.487  | 0.436    | 0.601  | 0.334  | 5.17  | 0.384     | 4.95  |
| UCB1              | 0.512              | 0.514  | 0.450    | 0.634  | 0.482  | 1.00  | 0.384     | 10.00 |
| REINFORCE-MLP     | 0.513              | 0.514  | 0.452    | 0.636  | 0.483  | 1.00  | 0.391     | 1.00  |
| PPO               | 0.512              | 0.509  | 0.452    | 0.646  | 0.464  | 1.59  | 0.390     | 1.66  |
| VideoAgent (ref.) | 0.713              | n/a    | n/a      | n/a    | n/a    | ~8    | n/a       | n/a   |
| EVA, GRPO (ref.)  | 0.60 <sup>†</sup>  | n/a    | n/a      | n/a    | n/a    | n/a   | n/a       | n/a   |

Table 2: Decision-isolation ablation on NExT-QA. Return and mean tool calls for each policy under each mask. Stopping-only nearly matches joint; tool-choice-only collapses. UCB1 and REINFORCE are means over three seeds; PPO uses one seed for the masked variants.

| Policy        | Joint  |       | Tool-choice only |       | Stopping only |       |
|---------------|--------|-------|------------------|-------|---------------|-------|
|               | Return | Calls | Return           | Calls | Return        | Calls |
| UCB1          | 0.482  | 1.00  | 0.223            | 9.71  | 0.486         | 1.00  |
| REINFORCE-MLP | 0.483  | 1.00  | 0.222            | 9.74  | 0.485         | 1.01  |
| PPO           | 0.464  | 1.59  | 0.227            | 9.74  | 0.465         | 0.85  |

trail it slightly, because random gathers more frames, just wastefully, and the frozen reader barely cares which frames it gets. The learners’ whole advantage is that they gather one useful frame and then stop, where random spends about five calls reaching the same answer. What the learning buys here is efficiency, not better answers, and the only decision worth learning is when to stop. The big gap to VideoAgent’s published 71.3% is expected, and honestly it is the point: that system reads frames with GPT-4, while I read them on purpose with a cheap CLIP oracle, so the comparison stays about the controller and not the reader. The last row, EVA, is the modern GRPO recipe I am implicitly arguing against; it reports on harder long-video benchmarks rather than NExT-QA, so it anchors where that recipe lands rather than offering a head-to-head.

The flat accuracy nagged at me, so I poked at the answer function directly. With no frames at all, comparing the question to the options scores about 0.30. Hand it essentially any frames and it climbs to about 0.47, and adding the question-conditioned term gets it to about 0.50. Then I asked whether which frames you pick matters, and this is where I had to correct my own first impression. Under mean-pooling, which averages the chosen frames into one vector, the four most query-relevant frames (0.475) beat the four least relevant (0.454) by only about two points. But the gap roughly doubles, to four or five points, when the reader scores a single best frame instead of a pooled set, and the same ordering shows up on EgoSchema. So gathering frames helps a lot, choosing the right ones helps a little, and the pooling I use hides even that little. Because the agent answers from a mean-pooled vector, its accuracy is set mostly by this weak reader rather than by which frames it gathers, and the only thing left for the controller to improve is the return.

### 5.3 Decision isolation

Next I asked which decision the learning was actually buying, using the masked settings. Table 2 is blunt about it. When the agent controls only stopping, the return is essentially the same as the full problem, around 0.48 at one call. When it controls only tool choice and the environment handles stopping, the return collapses to about 0.22, because the fixed stopping rule makes it spend almost the whole budget. Since accuracy barely moves across the three settings, that collapse is entirely the

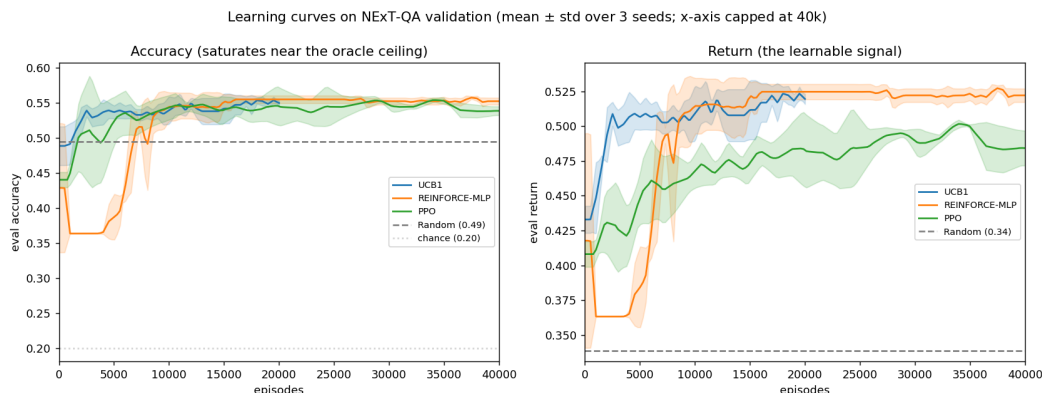


Figure 2: Learning curves on NExT-QA validation, mean and standard deviation over three seeds. Left: accuracy saturates near the oracle ceiling for every learner. Right: return, the learnable signal, rises well above the random baseline, and the bandit (UCB1) converges sooner than PPO. The x-axis is capped at 40k episodes, where all three have converged; PPO trains much longer but its curve is flat beyond.

price of extra calls. So the lesson is simple: learning when to stop is what matters here, and learning which tool to call is not, at least while the reader is this insensitive to which frames it gets.

## 5.4 Sample efficiency

My third question was how fast each learner gets there, and this is where averaging over seeds and using a finer measurement grid changed my answer. Figure 2 plots accuracy and return against training episodes. Accuracy saturates near the oracle ceiling almost immediately for everyone, so the interesting panel is return. If I mark convergence as the episode where the smoothed return first reaches 95% of its final value, the bandit gets there in about 3,700 episodes and PPO in about 11,800, averaged over three seeds. So the bandit converges roughly three times sooner than PPO, not the order of magnitude that an earlier single-seed run on a coarser grid had suggested. REINFORCE lands in between, around 7,800, but its convergence point swings wildly across seeds, from 500 to 15,500 episodes, so I would not read much into its exact value. One honest caveat on all of these: they are environment episodes, not wall-clock time or compute, and a bandit’s update is far cheaper per step than PPO’s. But since the real cost of training one of these agents is the model call on every step, episodes are the budget that would actually bite.

## 5.5 Transfer and qualitative behavior

What about transfer? On EgoSchema both learned networks carry their stopping habit over cleanly: REINFORCE scores 0.391 answering in one call, and PPO 0.390 in about one and a half. The UCB1 lookup table does not. EgoSchema’s question category is one it never saw, so the table falls back to a default bucket and the agent spends all ten calls, and even then only reaches 0.384. That contrast is a small, satisfying lesson in why a network generalizes where a lookup table cannot. I see little sign of overfitting either; training and validation accuracy track each other closely for every learner. Behaviorally, all of them converge to the same boringly sensible habit, gather one focused batch of frames and then answer. UCB1 does it in exactly one call on all 4,996 validation questions. The hardest category is temporal (about 0.45), which makes sense because averaging the frame vectors throws away their order, and the easiest is descriptive (about 0.64).

## 5.6 Raising the ceiling

Before stopping I tried two cheap ways to lift the ceiling, since that is where all the headroom is. Calibrating the oracle, by adding a question prior and subtracting a generic average-video vector, buys about a point on the probe and nothing once a policy already answers in one call. Swapping the frozen CLIP backbone for a much larger one, ViT-H/14 in place of ViT-L/14, does not help either: the ceiling stays near 0.51 and the frame-insensitivity is still there. That was the most useful negative

result for me. It says the limit is the contrastive matching idea itself, not the size of the model, so the next step has to be a different kind of reader, not a bigger one.

## 6 Conclusion and Future Work

With the reader held fixed, the story is short. A simple bandit recovers the quality of PPO, if anything edging ahead on return because it answers in fewer calls, while learning about three times faster; the stopping decision is where the learnable value lives, and accuracy is set by the frozen reader rather than the controller. These are the controlled baselines the recent reinforcement-learning-for-video papers skip over, and they point to a concrete habit: before spending GRPO-scale compute, check whether a simple learned stopping rule already captures the gain, and check whether the reader is the real bottleneck. The setup here is a cheap way to run exactly those checks, and the finding that a bigger backbone does not move the ceiling is itself a useful signpost for where not to spend effort.

### 6.1 Limitations

This is a small study and it has real limits. I average each configuration over three seeds, which is enough to show that the accuracy and return numbers are stable but not enough to pin down the noisier ones, like REINFORCE’s convergence episode, and the masked PPO ablations use a single seed, so I read those only for direction. The reader is cheap on purpose, so the absolute accuracy sits far below anything that answers with a large language model, and the numbers here are not meant to be state of the art. Averaging the frames into one vector throws away their order, which caps the temporal questions, and it also hides how much frame choice matters: picking the right frames helps about twice as much when the reader scores a single frame instead of a pooled one, so the dormant tool-choice axis is partly an artifact of pooling rather than proof that frame choice is useless. And the EgoSchema transfer is zero-shot with different clip lengths and window meanings, so a gap there could be poor generalization or just shifted action semantics, and I cannot fully separate the two.

### 6.2 Future work

The clearest next step is to swap the contrastive oracle for a frozen reader that actually reasons, such as BLIP-2 used in question-answering mode. That should lift the ceiling and, just as important, make frame choice matter again, which would wake up the tool-choice axis that sits dormant under the current reader. After that I would give the evidence some temporal structure so order-sensitive questions improve, add more seeds to tighten the few numbers that are still noisy, and only then ask whether GRPO buys anything on top of these classical baselines. The value of this study is not the final accuracy, which is low by design. It is that it turns those future comparisons into something you can actually read.

## References

- Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47:235–256.
- Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning (ICML)*.
- Karttikeya Mangalam, Raiymbek Akshulakov, and Jitendra Malik. 2023. Egoschema: A diagnostic benchmark for very long-form video language understanding. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Alec Radford, Jong Wook Kim, Chris Hallacy, et al. 2021. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Zhihong Shao et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

- Shulin Tian et al. 2025. Ego-r1: Chain-of-tool-thought for ultra-long egocentric video reasoning. *arXiv preprint arXiv:2506.13654*.
- Xiaohan Wang, Yuhui Zhang, Orr Zohar, and Serena Yeung-Levy. 2024. Videoagent: Long-form video understanding with large language model as agent. In *European Conference on Computer Vision (ECCV)*.
- Ronald J. Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256.
- Junbin Xiao, Xindi Shang, Angela Yao, and Tat-Seng Chua. 2021. Next-qa: Next phase of question-answering to explaining temporal actions. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Zhuo Yang et al. 2025. Videoagent2: Enhancing the llm-based agent system for long-form video understanding by uncertainty-aware cot. *arXiv preprint arXiv:2504.04471*.
- Yaolun Zhang, Ruohui Wang, Jiahao Wang, et al. 2026. Eva: Efficient reinforcement learning for end-to-end video agent. *arXiv preprint arXiv:2603.22918*.
- Yiyang Zhou et al. 2025. Reagent-v: A reward-driven multi-agent framework for video understanding. In *Advances in Neural Information Processing Systems (NeurIPS)*.